

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – Coding Challenge Task

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO4: Articulation (BL4, BL5)	Assessment:	Assessment Tool 1 – Coding Challenge Task
Weightage:	40% of CIA	Total Marks:	100
CO4 Statement:	Solve optimization problems using dynamic programming and greedy strategies.		
Student Name:		Reg. No.:	
Section / Batch:		Date:	

Instructions to Students

- Answer the following question. Total Marks: 100.
- Carefully analyze the given questions.
- Write clear code for the given problem.
- Analyze the Time complexity and Space complexity of your code using dynamic programming and greedy strategies

Question Type	Focus Area	Questions
Coding Challenge Task	Focus on Code and analyze optimization problems using dynamic programming and greedy strategies	Q1

SECTION A – Coding Challenge Task

Q20. Maximum Sum Subarray

Given an array of integers containing both positive and negative values, find the contiguous subarray that has the largest possible sum. The chosen elements must appear consecutively in the original array. The objective is to determine the maximum sum that can be obtained from any such subarray. Use Dynamic Programming to compute the optimal result efficiently.

Sample Input:

n = 5

arr = -2 -3 4 -1 -2

Expected Output:

Maximum Sum = 4

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

RUBRICS FOR CALCULATION

Coding Challenge Task

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly understands problem constraints, edge cases, and competitive requirements	Understands problem with minor gaps in constraints	Partial understanding; misses key constraints	Misinterprets problem or constraints
Algorithm	25%	Selects optimal algorithm with best time and space complexity for competitive constraints	Uses efficient algorithm with minor scope for improvement	Uses basic/inefficient approach	No clear algorithmic approach
Code Implementation	20%	Writes correct, efficient, and clean code that passes all constraints	Code works with minor errors or inefficiencies	Partially correct code; fails for some cases	Code does not execute or gives incorrect results
Competitive Performance	20%	Solves problem within time limits and handles large inputs effectively	Solves within acceptable time with minor delays	Struggles with time limits or larger inputs	Unable to solve within time constraints
Test Case Handling	15%	Handles all test cases including edge and hidden cases	Handles most test cases	Misses important edge cases	Fails on multiple test cases

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%				
Algorithm	25%				
Code Implementation	20%				
Competitive Performance	20%				
Test Case Handling	15%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name: _____	Name: _____	Name: _____
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

1. Introduction

The **Maximum Sum Subarray** problem is a common Dynamic Programming problem. It requires finding a contiguous part of an array whose elements produce the largest possible sum. The elements in the selected subarray must occur consecutively in the original array. Dynamic Programming can solve this problem efficiently by keeping track of the maximum sum that can be obtained up to each position in the array.

2. Problem Statement

Given an array containing both positive and negative integers, find the **contiguous subarray with the maximum sum**.

The objective is to determine the maximum sum that can be obtained from any consecutive group of elements.

Sample Input

n = 5

arr = -2 -3 4 -1 -2

Expected Output

Maximum Sum = 4

3. Problem Analysis

A subarray must contain **consecutive elements** from the original array.

For the given array:

-2 -3 4 -1 -2

Some possible subarrays are:

- [-2] → -2
- [-3] → -3
- [4] → 4
- [4, -1] → 3
- [4, -1, -2] → 1
- [-2, -3] → -5

The largest sum is obtained from the subarray:

[4]

Therefore:

Maximum Sum = 4

4. Dynamic Programming Approach

The problem can be solved using **Kadane's Algorithm**, which is a Dynamic Programming technique.

For every element, we decide whether:

1. To **extend the previous subarray**, or
2. To **start a new subarray** from the current element.

The recurrence relation is:

$$dp[i] = \max(arr[i], dp[i-1] + arr[i])$$

Where:

- $dp[i]$ = maximum sum of a subarray ending at position i
- $arr[i]$ = current element

The final answer is the maximum value among all $dp[i]$.

5. Step-by-Step Analysis

Given:

$arr = [-2, -3, 4, -1, -2]$

Index Element Maximum Subarray Sum Ending Here

0	-2	-2
1	-3	-3
2	4	4
3	-1	3
4	-2	1

The maximum value in the table is:

4

Thus, the maximum sum subarray is:

[4]

6. Algorithm

1. Initialize currentSum with the first element.
2. Initialize maxSum with the first element.
3. Traverse the array from the second element.
4. For each element, calculate:
5. $\text{currentSum} = \max(\text{arr}[i], \text{currentSum} + \text{arr}[i])$
6. Update:
7. $\text{maxSum} = \max(\text{maxSum}, \text{currentSum})$
8. After processing all elements, maxSum contains the maximum subarray sum.
9. Display the result.

7. Pseudocode

Algorithm MaximumSumSubarray(arr, n)

currentSum = arr[0]

maxSum = arr[0]

for i = 1 to n-1 do

 currentSum = max(arr[i], currentSum + arr[i])

 maxSum = max(maxSum, currentSum)

end for

return maxSum

8. Working Example

For the input:

-2 -3 4 -1 -2

Initially:

currentSum = -2

maxSum = -2

For -3:

$\text{currentSum} = \max(-3, -2 + -3)$

$= -3$

$\text{maxSum} = -2$

For 4:

$\text{currentSum} = \max(4, -3 + 4)$

$= 4$

$\text{maxSum} = 4$

For -1:

$\text{currentSum} = \max(-1, 4 + -1)$

$= 3$

$\text{maxSum} = 4$

For -2:

$\text{currentSum} = \max(-2, 3 + -2)$

$= 1$

$\text{maxSum} = 4$

Therefore:

Maximum Sum = 4

8. Complexity Analysis

Time Complexity

The array is traversed only once.

Time Complexity = $O(n)$

Space Complexity

Only two variables are required to store the current and maximum sums.

Space Complexity = $O(1)$

This makes the Dynamic Programming approach highly efficient for large arrays.

9. Advantages

- Uses an efficient Dynamic Programming approach.
- Requires only one traversal of the array.
- Works with both positive and negative integers.

- Uses constant extra space.
- Faster than checking every possible subarray.

10. Conclusion

The Maximum Sum Subarray problem can be efficiently solved using **Dynamic Programming with Kadane's Algorithm**. Instead of calculating the sum of every possible subarray, the algorithm maintains the best sum ending at each position. For the given input [-2, -3, 4, -1, -2], the maximum sum is **4**, obtained from the subarray [4]. The algorithm achieves **$O(n)$ time complexity and $O(1)$ space complexity**, making it suitable for large datasets.

11. Code

```
def max_subarray_sum(arr):
    max_sum = arr[0]
    current_sum = arr[0]

    for i in range(1, len(arr)):
        current_sum = max(arr[i], current_sum + arr[i])
        max_sum = max(max_sum, current_sum)

    return max_sum

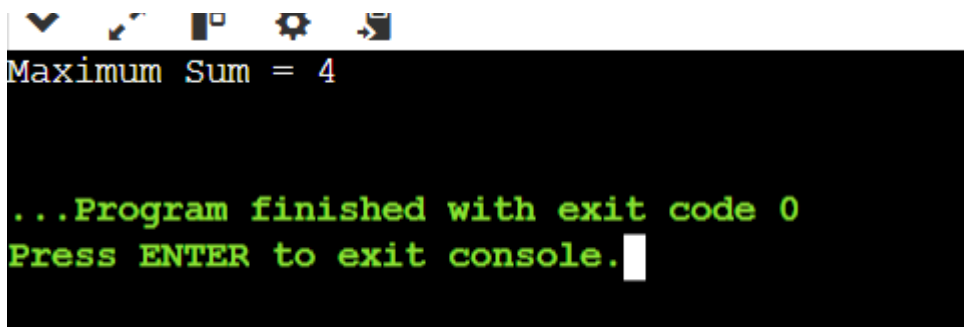
n = 5
arr = [-2, -3, 4, -1, -2]

print("Maximum Sum =", max_subarray_sum(arr))
```

main.py

```
1 def max_subarray_sum(arr):
2     max_sum = arr[0]
3     current_sum = arr[0]
4
5     for i in range(1, len(arr)):
6         current_sum = max(arr[i], current_sum + arr[i])
7         max_sum = max(max_sum, current_sum)
8
9     return max_sum
10
11 n = 5
12 arr = [-2, -3, 4, -1, -2]
13
14 print("Maximum Sum =", max_subarray_sum(arr))
15
```

Sample output-



Maximum Sum = 4

...Program finished with exit code 0
Press ENTER to exit console.

Explanation

1. Input the array

The given array is [-2, -3, 4, -1, -2] and it contains 5 elements.

2. Initialize the variables

current sum and max sum are initialized with the first element -2. Current sum stores the best sum ending at the current position, while max sum stores the overall best sum.

3. Traverse the array

The for loop checks each element from the second element to the last element.

4. Find the current maximum

For each element, the program compares the current element alone with the sum of the previous subarray and the current element. It keeps whichever is larger.

5. Update the maximum sum

After calculating current sum, the program compares it with max sum and stores the larger value.

6. Get the final result

For the given array, the largest sum is obtained from the subarray [4].

7. Final Answer

Therefore, the maximum sum is **4**.

Complexity

- **Time Complexity:** $O(n)$
- **Space Complexity:** $O(1)$